

 ATlassian

atlas camp

Goodbye DC, Hello Forge -

Building for a Cloud-First Future

KARTHIKEYAN SREEKUMAR

PAUL T GEORGE

The DC Reality We're Leaving Behind

4-step EOL timeline



04

Mar 28 2029

Final EOL. All DC products and apps enter read-only



03

Mar 30 2028

Final date for existing customers to purchase new products or expand existing user tiers.



02

Mar 30 2026

End of sales for new Data Center customers.

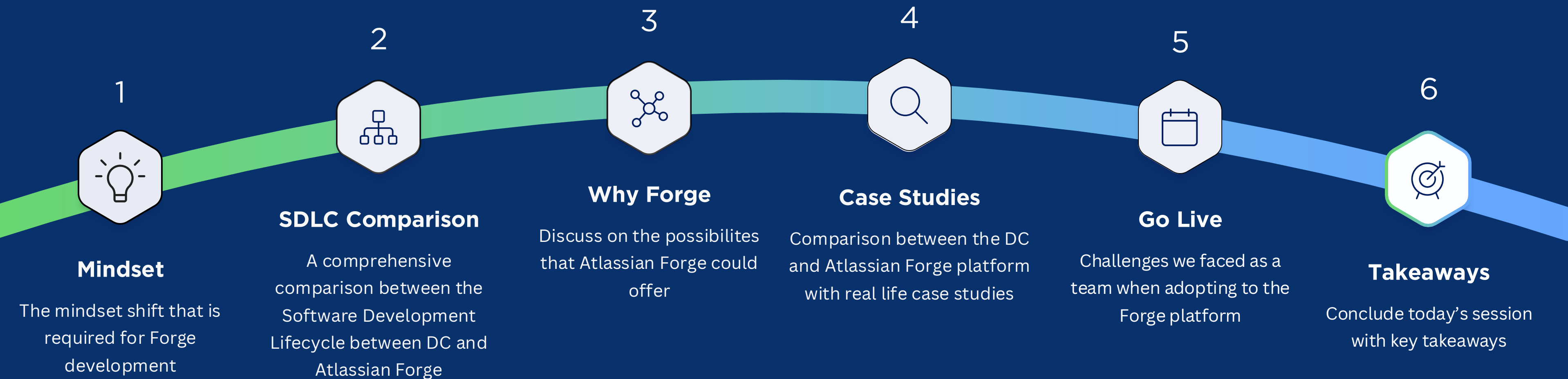


01

Dec 16 2025

No new Data Center Marketplace apps accepted

The Forge Journey: Roadmap



A New Mindset for the Forge Platform

DC

Inside the customer's clustered JVM — you manage memory, thread safety, node restarts

Active Objects on the customer's DB — you own schema & migrations

Local Jira via Plugin SDK, atlas-run/atlas-debug, JVM mismatch risk

Where Your Code Runs

Where Your Data Lives

How You Test & Debug

Forge

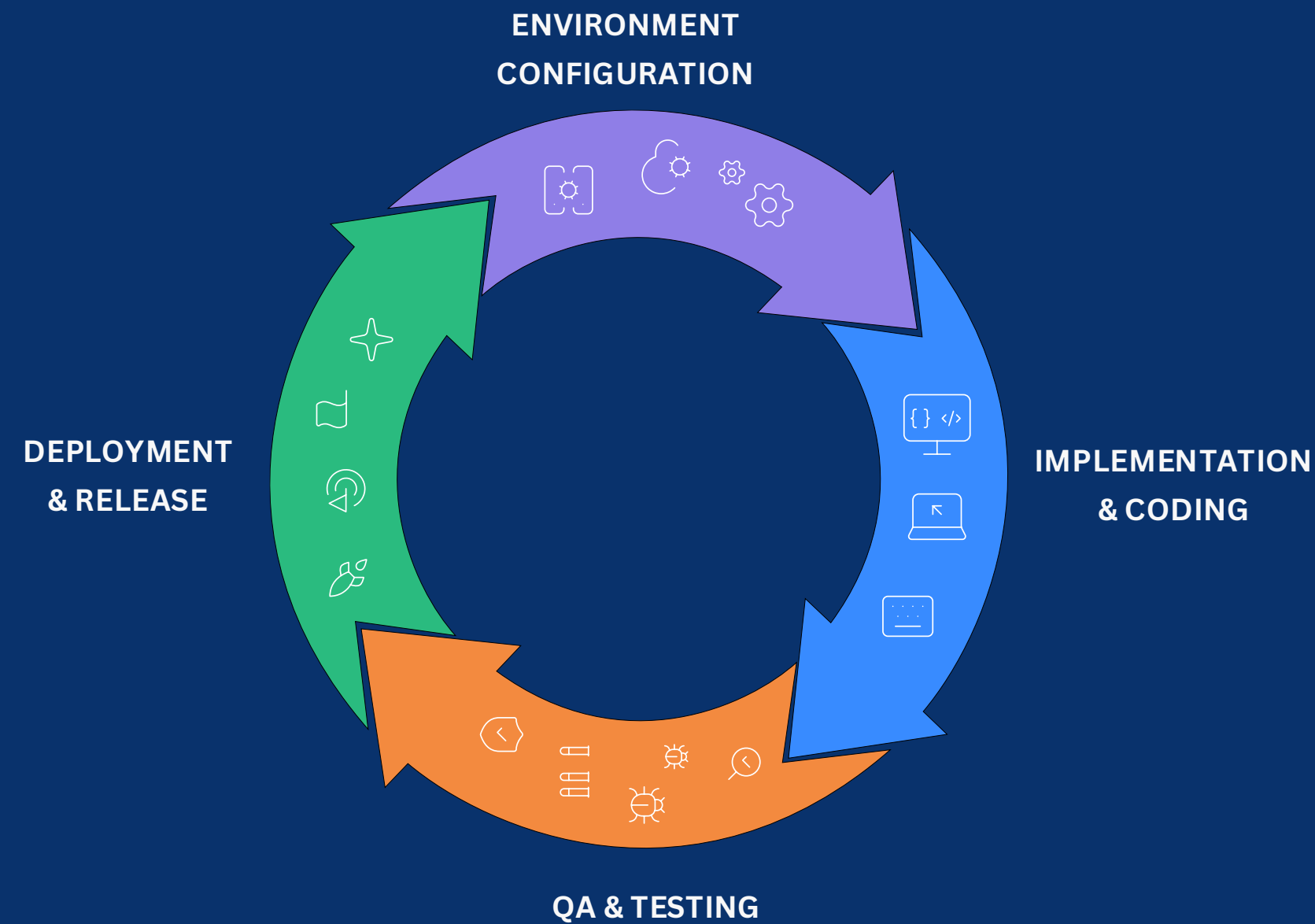
FaaS on AWS Lambda abstracted by Atlassian — think in invocations and cold starts

Atlassian-hosted: KVS, Custom Entity, Forge SQL — data residency built in

Forge tunnel wires local code to a live cloud site — instant browser feedback

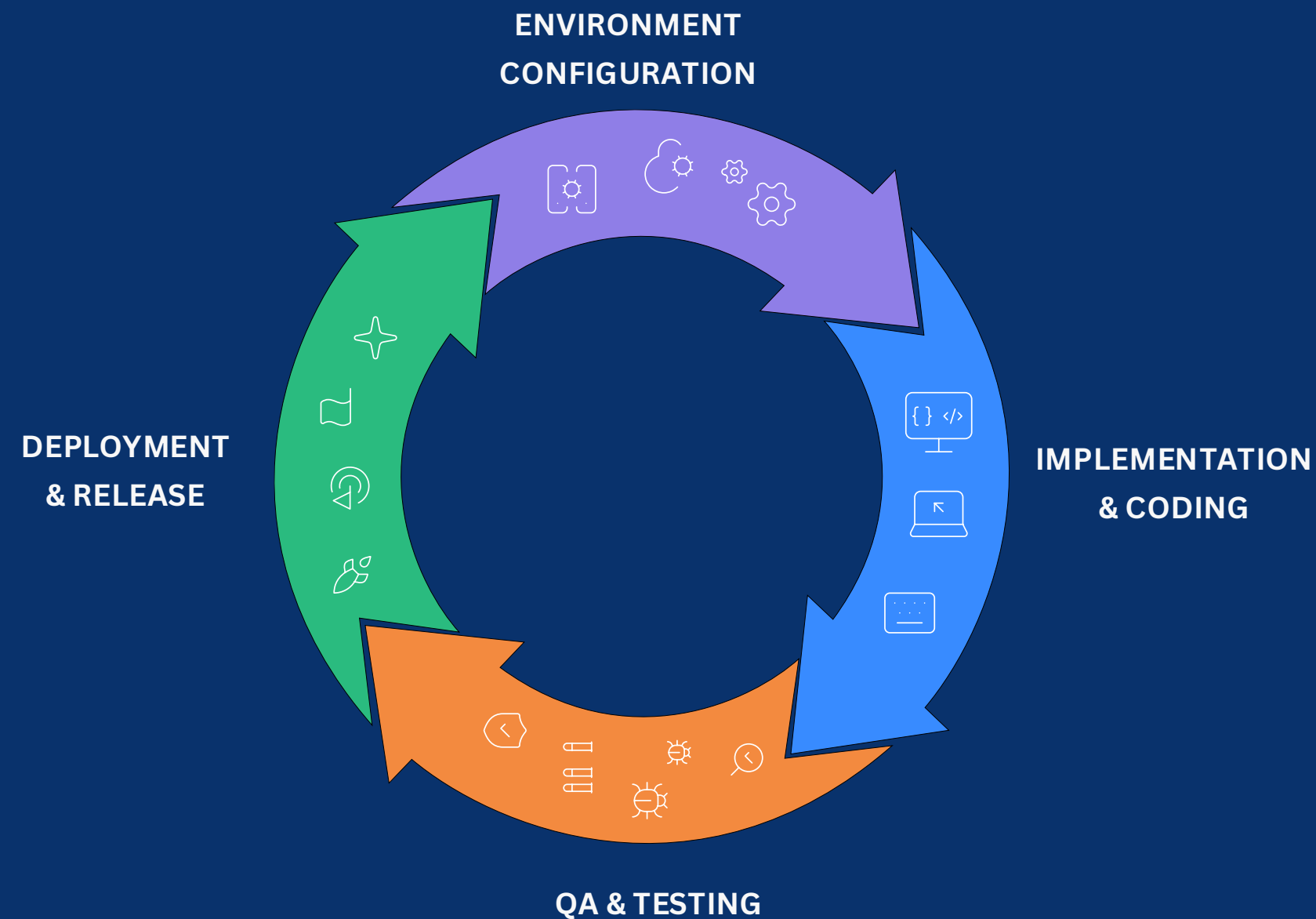
From DC to Forge: The SDLC Evolution

What basic software development looks like in Data Center



From DC to Forge: The SDLC Evolution

What basic software development looks like in Data Center

**01**

Environment Configuration

Java Sprint Boot and dependencies, along with Atlassian Jira SDK

02

Implementation & Coding

Develop cluster-aware applications by leveraging the Atlassian SDK to ensure thread safety and centralized data persistence.

03

QA & Testing

Validate app performance and reliability through rigorous testing against large-scale datasets and multi-node clusters

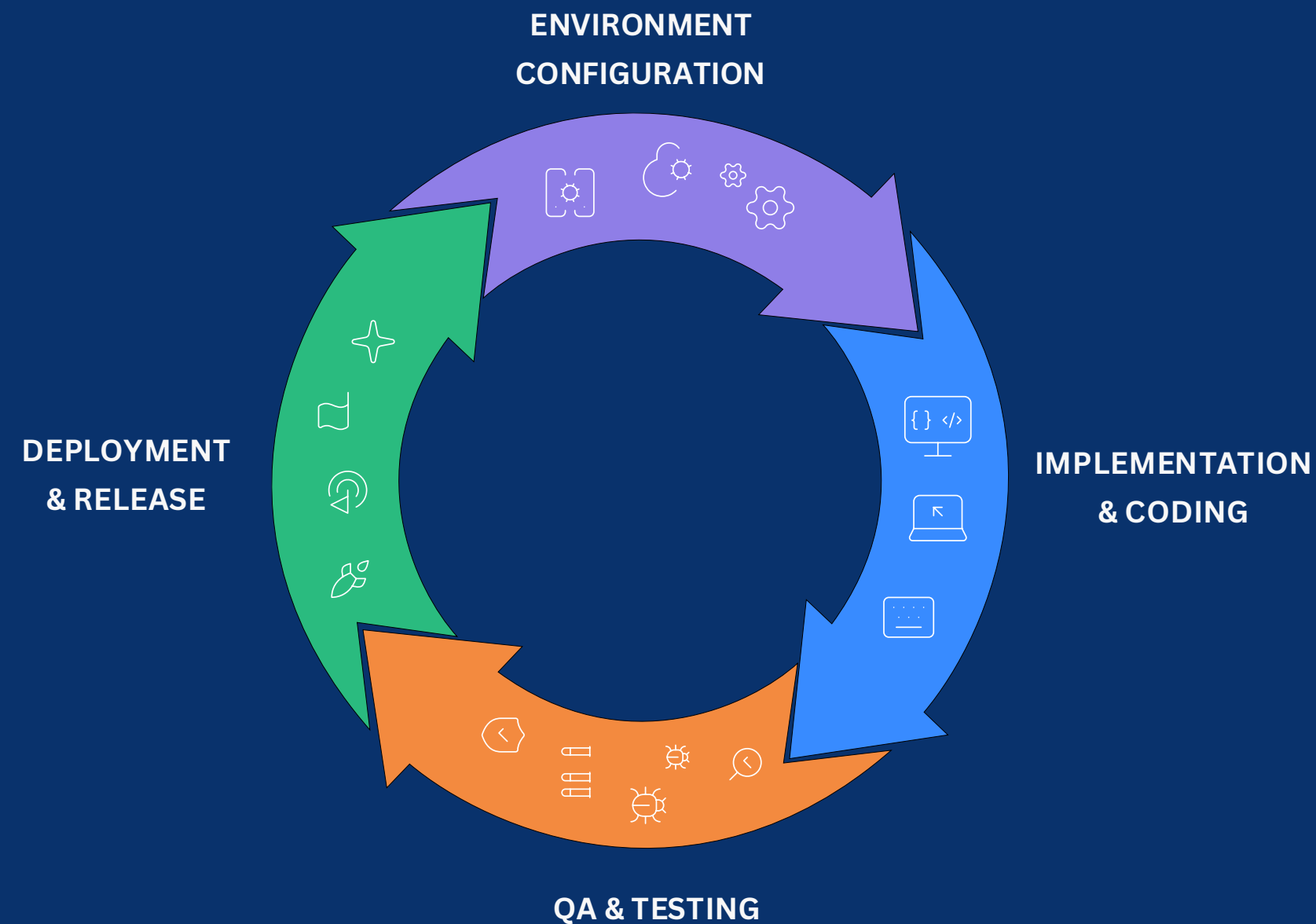
04

Deployment & Release

Submit the app for technical review and manage versioning via the Atlassian Marketplace for clustered distribution.

From DC to Forge: The SDLC Evolution

Set up the Forge CLI and initialize your project using standard templates for Jira

**01**

Environment Configuration

- The Forge CLI is the command line interface for building and deploying Forge apps.

```
npm i -g @forge/cli@latest
```

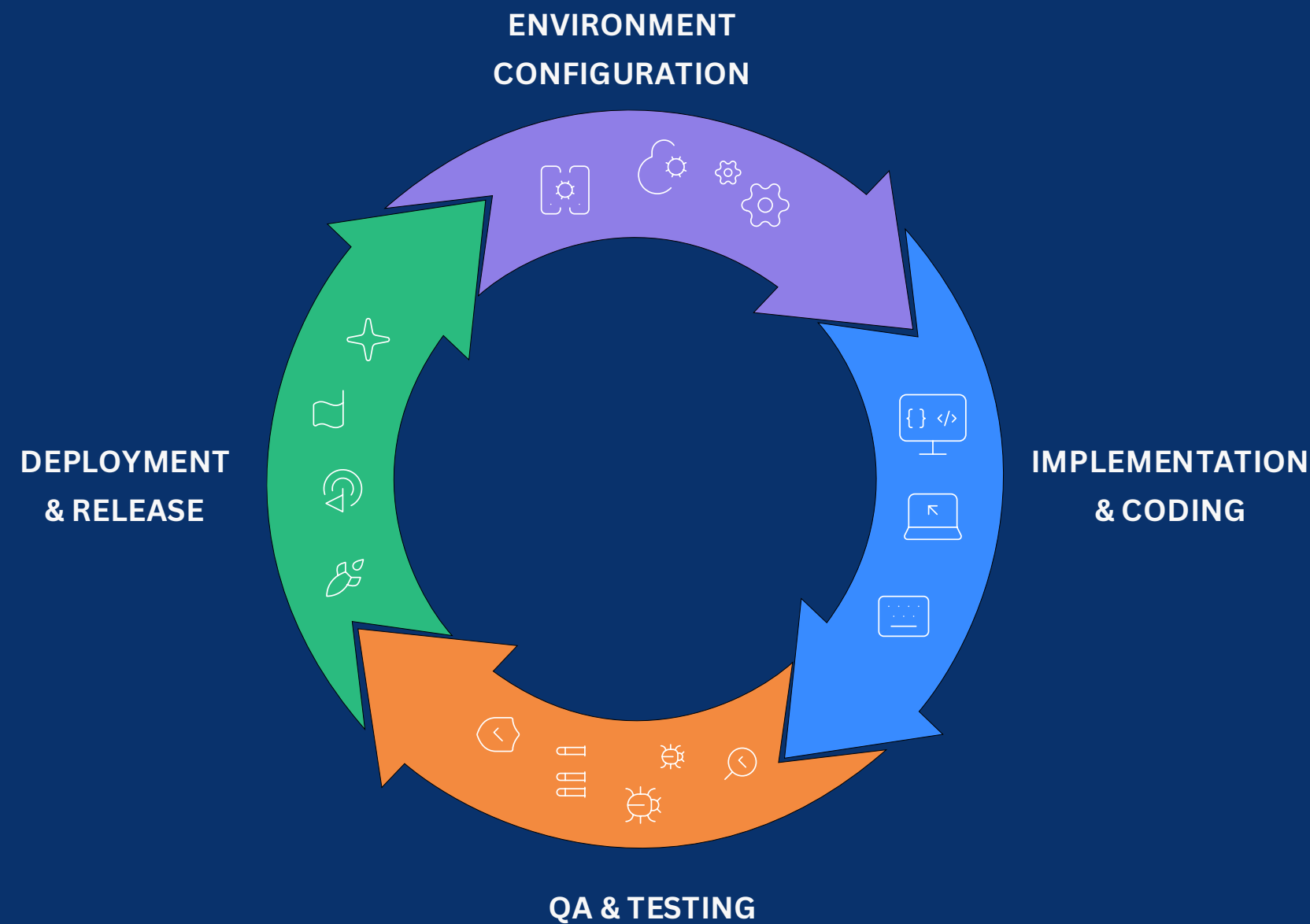
and log in using the command

```
forge login
```

- Three named environments out of the box
 - development
 - staging
 - production

From DC to Forge: The SDLC Evolution

Build secure apps using Forge's managed authentication and modern serverless storage solutions.



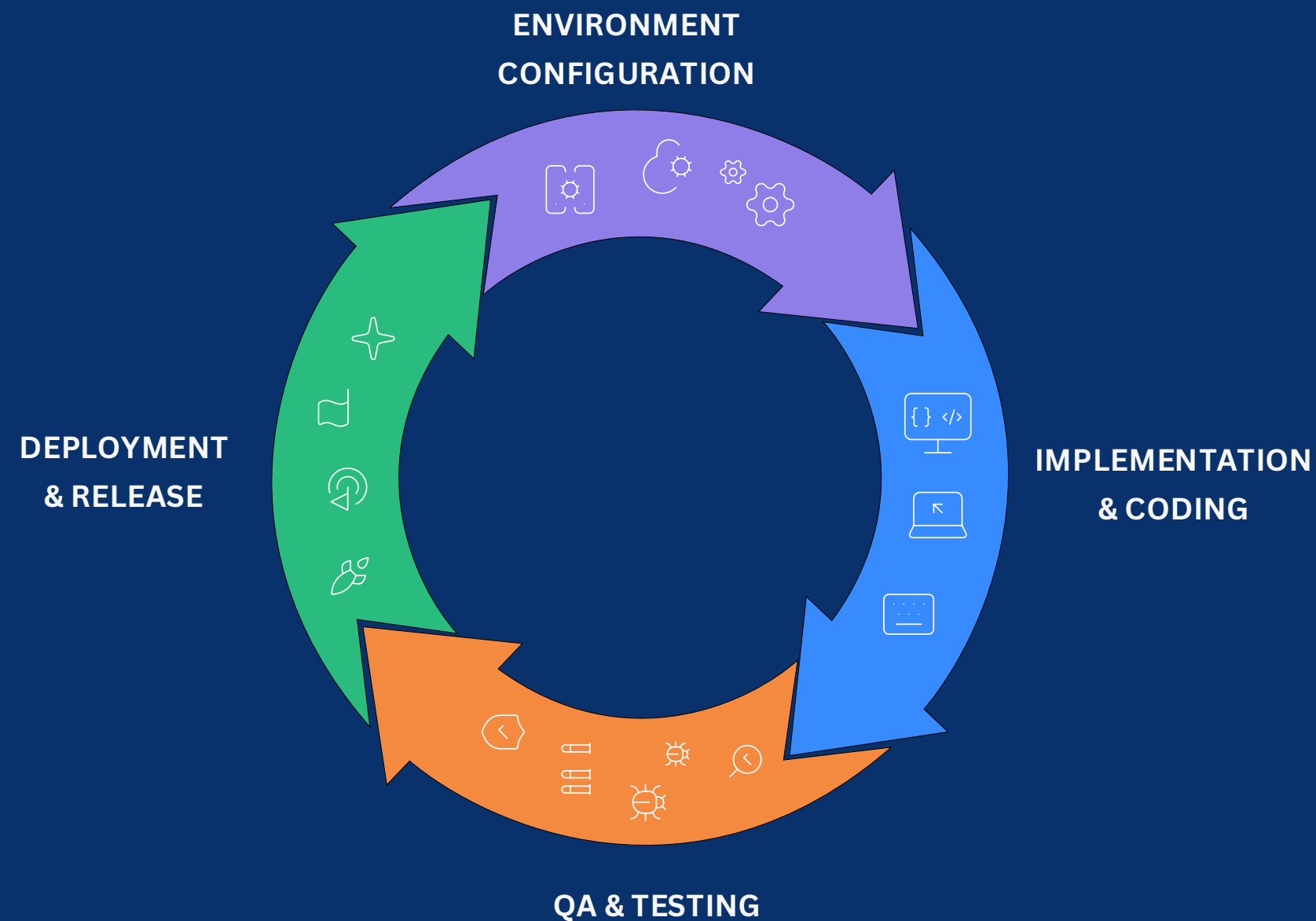
02

Implementation & Coding

- Managed Security
 - Runs on Atlassian → auth, context, permissions handled
 - Forge Remote → only your backend handles OAuth
- Data Persistence
 - Custom Entities → structured data
 - Forge SQL → relational queries
 - Object Store → files, blobs
- AI-Enhanced Development
 - Forge MCP Server for VS Code, Cursor or Rovo Dev CLI
- Automated Oversight
 - Code Reviewer Rovo Agent

From DC to Forge: The SDLC Evolution

Leverage real-time debugging and tiered environments to ensure app quality and performance.



03

QA & Testing

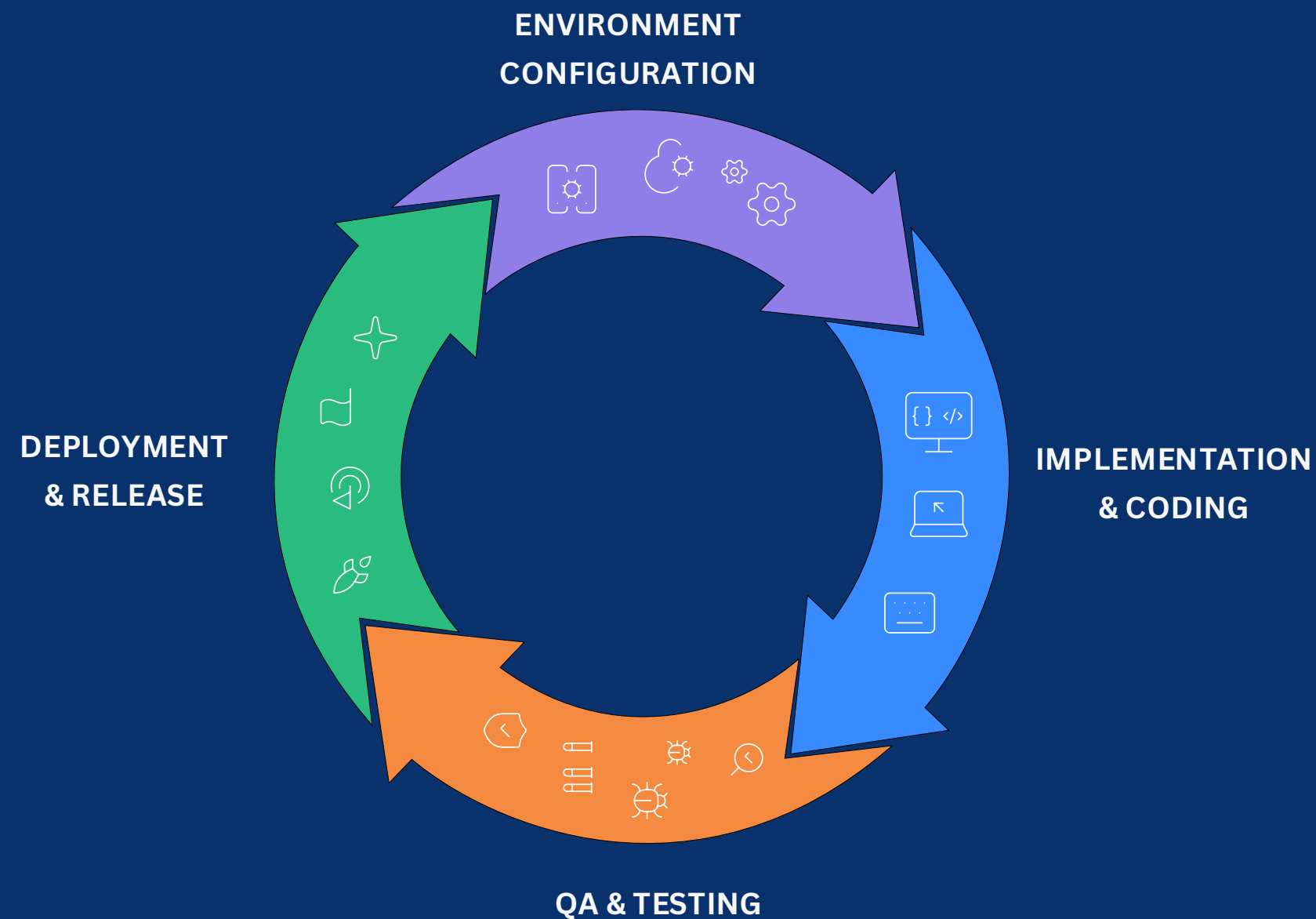
- Real-time Tunneling
 - Run code locally - but connect it to a live Atlassian site

```
forge tunnel -e <environment>
```
- Tiered Environments
 - 3 base environments, one command to deploy to each
- Linting


```
forge lint
```
- Unit and Integration Testing
 - Standard javascript tooling
- Scalability and Benchmarking
 - Function invocation timeouts
- Multi-user and Permission Testing
 - Need to test across different user permissions

From DC to Forge: The SDLC Evolution

How does Deployment & Release looks like in Atlassian Forge



04

Deployment & Release

- Multi-environment Promotion
 - forge deploy with an environment flag
 - dev → staging → production
- Zero-downtime Updates and App Management
 - Rolls out new versions without dropping active sessions
 - Installation metrics, real time logs, and error aggregation
- Marketplace Distribution
 - When you're ready to publish, link app to marketplace listing
 - Licensing is automated
- Data Residency and Compliance
 - Inherit SOC 2, ISO since app runs in Atlassian's security boundary
 - Automatic data residency

SDLC for DC vs Forge

DC

Forge

01

**Environment
Configuration**

Half-day setup with Java, Spring Boot, SDK, Maven

Minutes – CLI install, pick a template, 3 environments built in

02

**Implementation
& Coding**

Cluster-aware code, manual JWT/OAuth, Active Objects

Platform-managed auth & storage, AI tooling (Forge MCP, Rovo)

03

QA & Testing

Multi-node cluster validation, JVM mismatch risk

Real-time tunnel to live cloud site, instant feedback loop

04

**Deployment
& Release**

Package → marketplace review → customer-managed upgrades

Single CLI command, zero-downtime rollout, built-in data residency

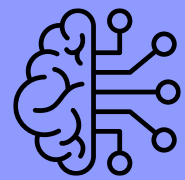
What does Forge let you build ?!

Lets see what's exciting about forge, and our experiences developing on Forge

What's exciting about Forge

Mindset of reimagining your apps which were not possible using DC

01



Forge LLMs

Use Atlassian-hosted LLMs to add secure AI features within the platform.

02



Teamwork Graph Connectors

Connect external data sources to Teamwork Graph for unified insights.

03



Forge Containers

Manage containerized services for Forge apps with built-in tools and capabilities.

04



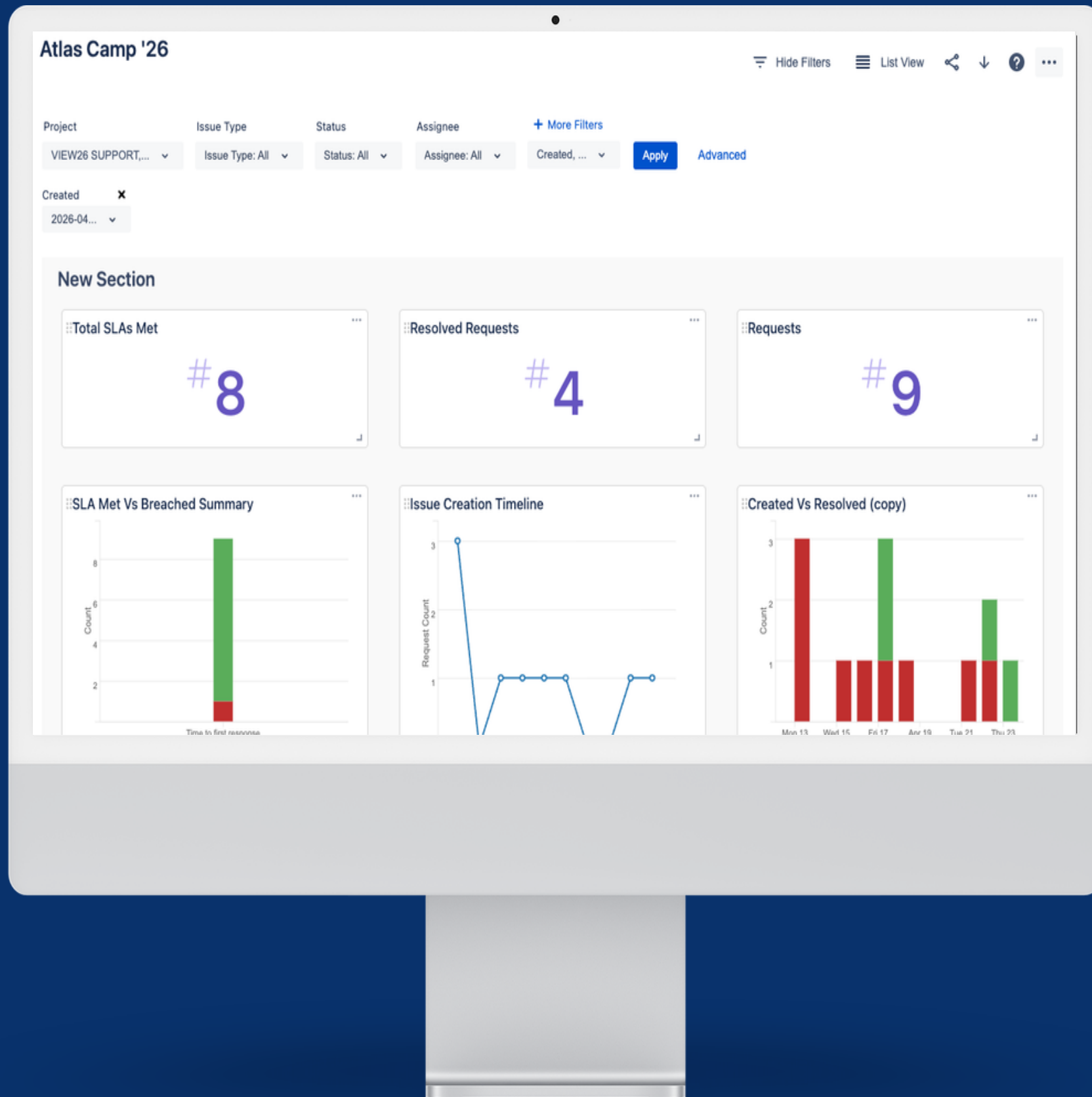
Object Store

Store, retrieve, and manage large data and media objects seamlessly within Forge apps.

Case Study 1: VIEW26 Charts and Reports for JSM

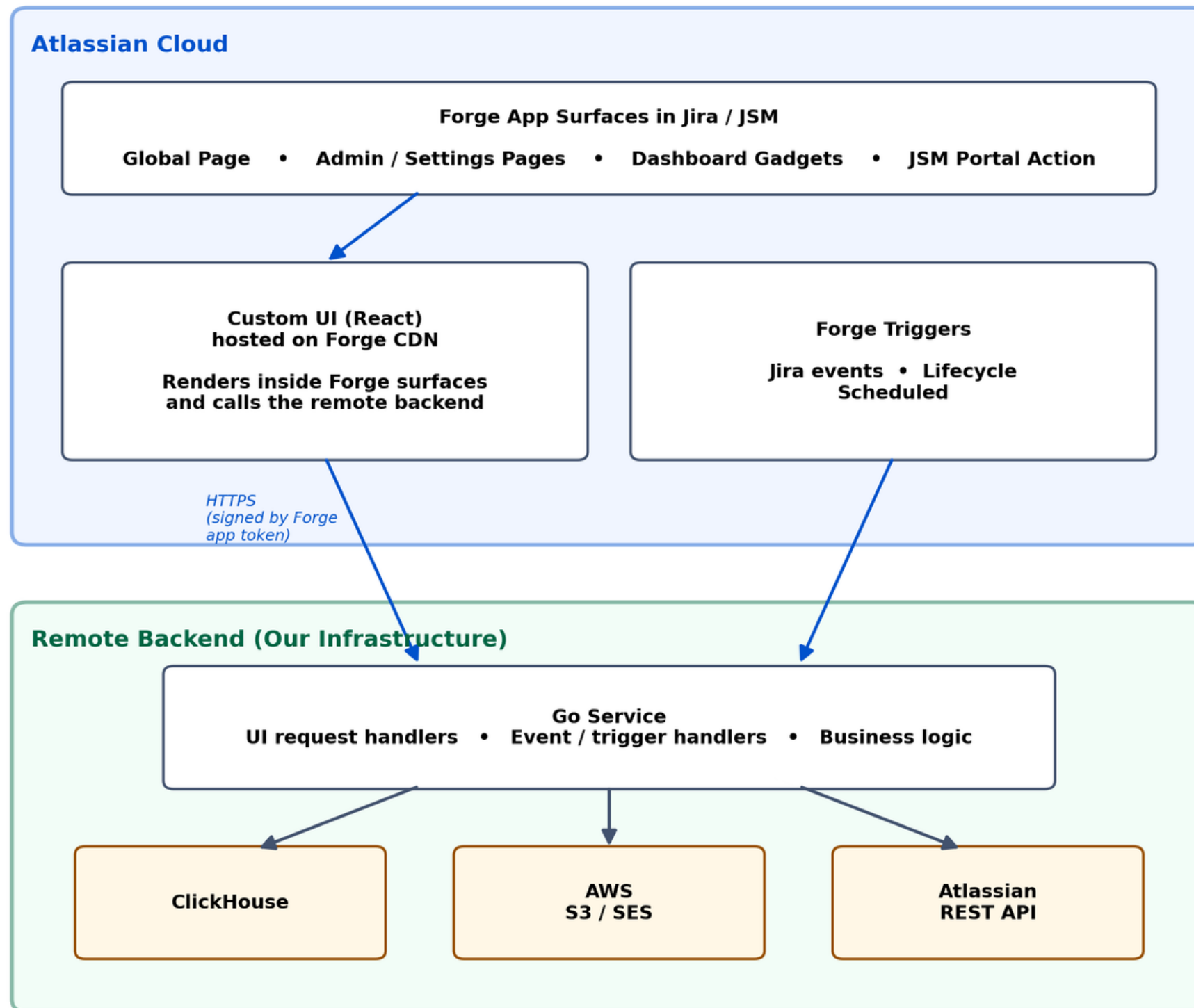
The Problem

- Heavy analytics and customer data
- Two platforms. Connect for Cloud. DC for the JVM.
- Two pipelines. Feature parity kept slipping.
- Then Connect hit sunset. DC wound down for new apps. Staying put wasn't an option.



Case Study 1: VIEW26 Charts and Reports for JSM

Forge Remote App — High-Level Architecture



What We Built and Why Forge Worked

- Heavy analytics on our infrastrucutre while everything else stayed on Forge.
- One codebase. One release cadence.
- Forge handled a bunch of platform and enterprise concerns
- Forge Remote lets you keep your backend without giving up on the platform's benefits.

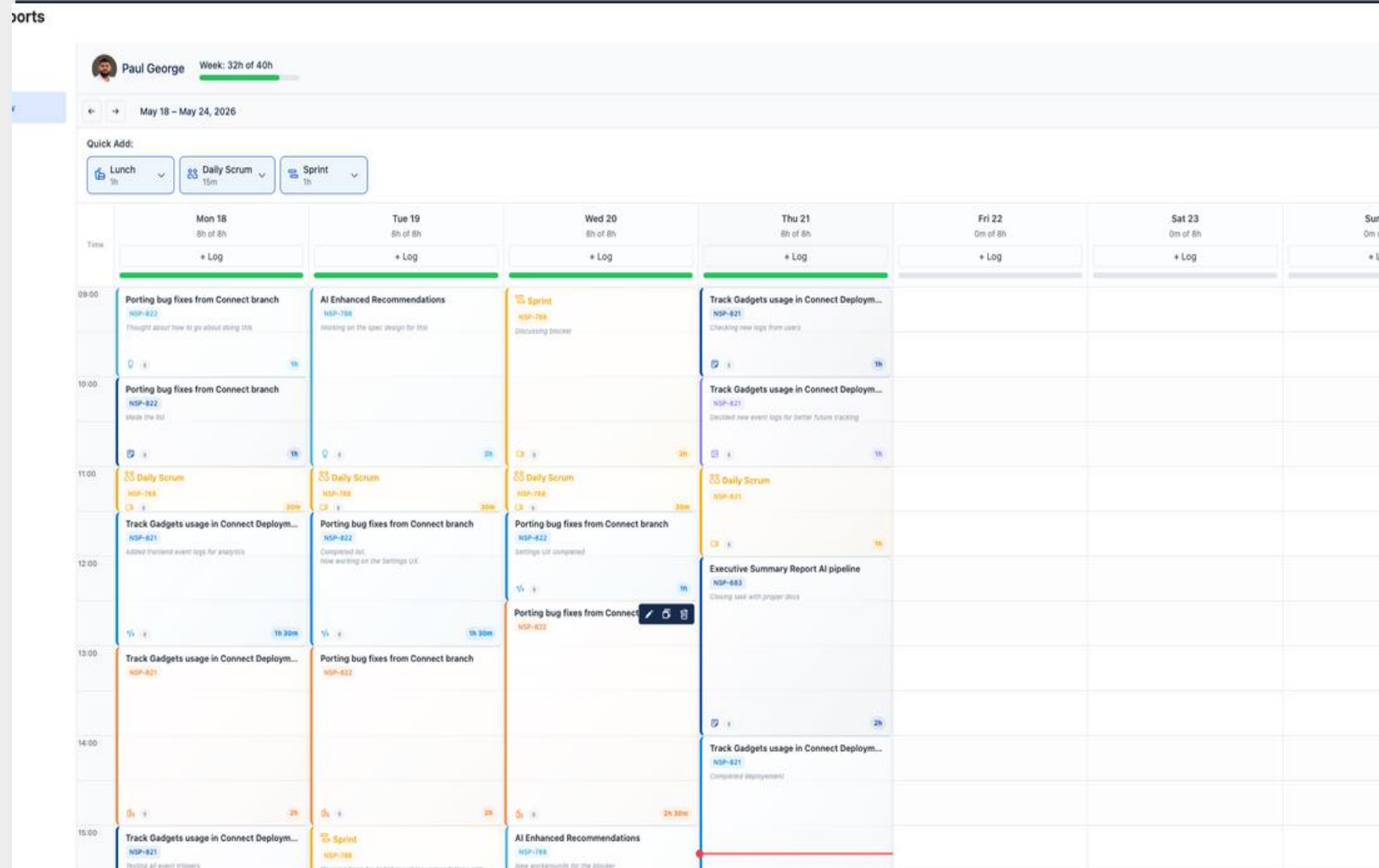
Case Study 2: VIEW26

Worklog Reports

Time Tracking & Timesheets for Jira

The Problem

- Brand new app developed on Forge
- Jira lets you log time, but doesn't organise it for easy use
- Jira does not provide work categories and reports

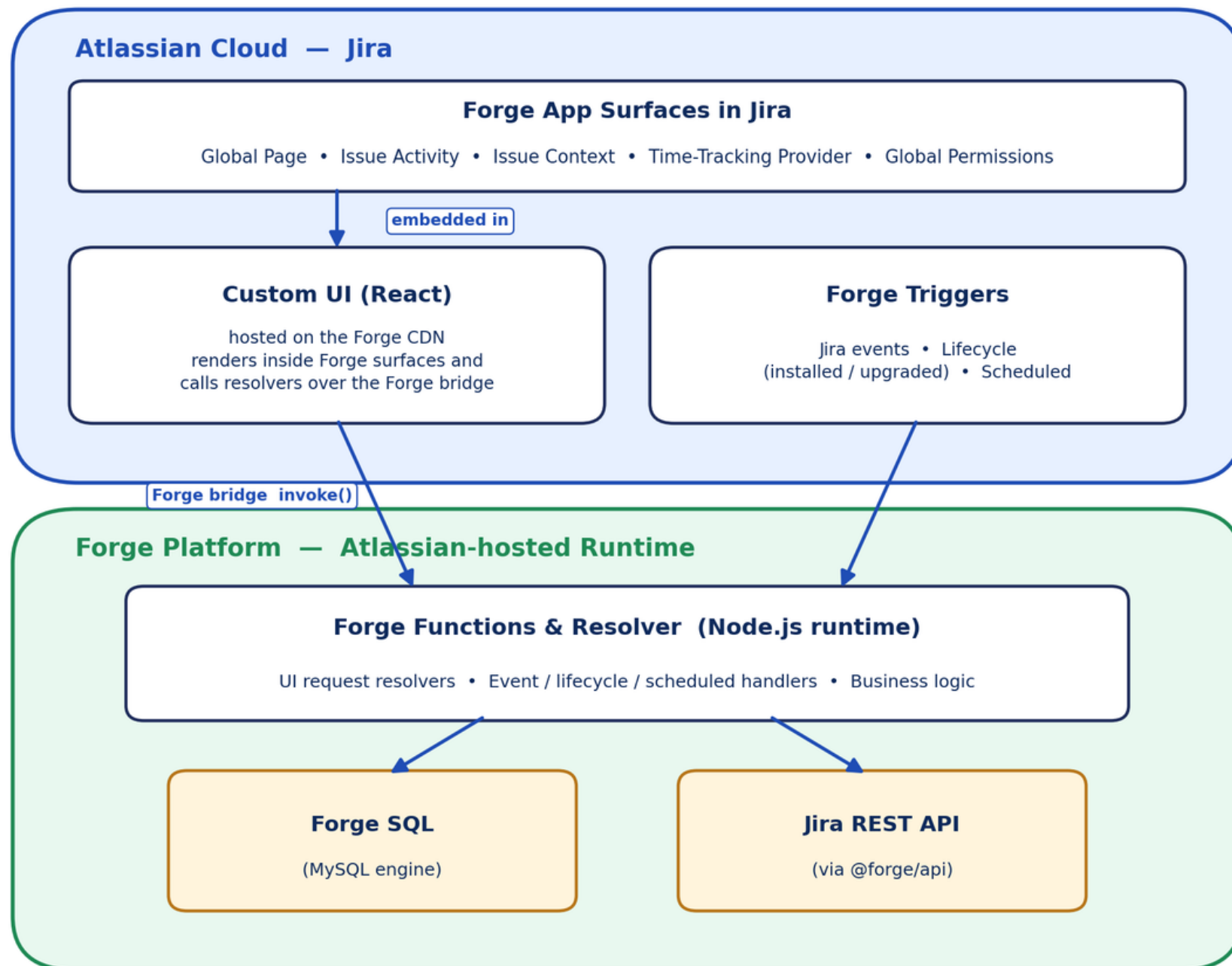


Case Study 2: VIEW26 view26

Worklog Reports

Time Tracking & Timesheets for Jira

Forge-Native App — High-Level Architecture



What We Built and Why Forge Worked

- Fully Forge Native - React Custom UI, Forge resolvers, and Forge SQL
- The app provides calendar-based time logging, work categories, reporting, and bidirectional sync with Jira
- Forge provides -
 - Forge SQL
 - Tenant isolation
 - Built-in migration system
 - Authentication: same API layer — just with different context for App and User
 - Bidirectional sync using Webhooks
 - Explicit permission declaration

Go-Live Challenges

Critical constraints and scalability hurdles encountered during production go-live phase

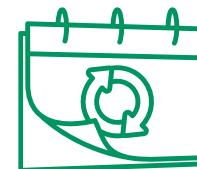
01

Resolver Timeouts (25s)



02

Forge Async Events



03

Forge SQL Limits



Go-Live Challenges

Critical constraints and scalability hurdles encountered during production go-live phase

01

Resolver Timeouts (25s)



Resolver Timeouts (25sec)

- We hit the 25-second Atlassian Forge resolver timeout limit
- Forced a redesign of backend logic to avoid failures in production
- Implemented the Async Events API to solve the issue

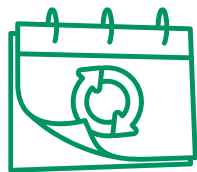
02

Go-Live Challenges

Critical constraints and scalability hurdles encountered during production go-live phase

02

Forge Async Events



Forge Async Events (15mins)

- 15-minute execution cap required building orchestration logic to reliably handle long-running tasks beyond a single invocation

03

Forge SQL Limits

03

Forge SQL Limits



Go-Live Challenges

Critical constraints and scalability hurdles encountered during production go-live phase

Forge SQL Limits

- Platform limits
 - Invocation limits and execution duration on functions,
 - Async event limits,
 - Resource bundle size limits for your UI,
 - KVS and Custom Entity Store limits,
- Object Store throughput limits, and
- LLM context window limits

03

Forge SQL Limits



Go-Live Challenges

Critical constraints and scalability hurdles encountered during production go-live phase

Forge SQL Limits

- Size limits:
 - Storage is per-installation, and the limit is tiered by environment
 - 1 GiB for production env
 - 256 MiB for staging env
 - 128 MiB for development env
- Beyond that:
 - 200 tables per install
 - DML queries get 5-10 second per connection timeouts
 - DDL queries get 20 second per connection timeouts
 - Request payload is 1 MiB, and response is 4 MiB

Key Takeaways

01



Strategy

Choose the right migration strategy

02



Track EAPs

Stay ahead with Forge EAPs

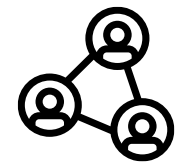
03



Think Beyond

Think beyond migration

04



Invest

Invest in team expertis

**Goodbye DC, Hello Forge -
Thank You, Bengaluru**
